Advanced Embedded Linux Image with Yocto_Upwilling

Integration Project Report

July 6, 2026

Chapter 1

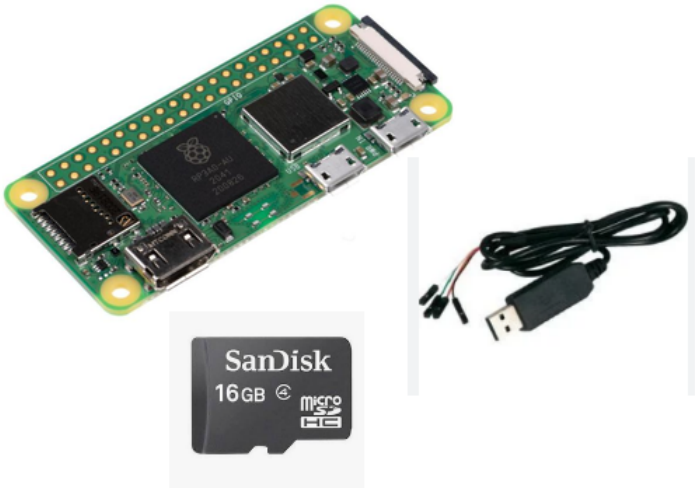
Introduction

1.1 Context

This project involves developing a Yocto-based embedded Linux image for the Raspberry Pi Zero 2 W. The objective is to design a robust, secure, and industrial-grade system foundation, incorporating advanced mechanisms such as OTA updates, A/B partitioning, and lightweight virtualization.

1.2 Hardware Requirements

- Raspberry Pi Zero 2 W
- USB to TTL Serial Cable (UART Debug)
- SD Card (16 GB)



1.3 Specifications Raspberrypi02w

- **Size:** 65 mm × 30 mm
- **Processor:** Broadcom BCM2710A1, 64-bit quad-core SoC (Arm Cortex-A53 at 1 GHz)
- **Memory:** 512 MB LPDDR2
- **Connectivity:**
  - IEEE 802.11b/g/n 2.4 GHz wireless LAN
  - Bluetooth 4.2, BLE, integrated antenna
  - USB 2.0 interface with OTG support
  - 40-pin GPIO header footprint compatible with HAT
  - microSD card slot
  - mini-HDMI port

Chapter 2

Project Setup

2.1 Host Environment Preparation

The following essential packages must be installed on the Ubuntu build host:

```
sudo apt install build-essential chrpath cpio debianutils diffstat file gawk gcc git \
iputils-ping libacl1 liblz4-tool locales python3 python3-git python3-jinja2 \
python3-pexpect python3-pip python3-subunit socat texinfo unzip wget xz-utils zstd
```

2.2 Kas Installation

Kas makes yocto builds simple, based on a YAML configuration file, kas starts a container, clones the layer repositories, initialises the conf files and starts building. Kas Documentation

```
git clone https://github.com/siemens/kas.git
```

2.3 Clone Layers and Prepare Configuration

```
kas/kas-container checkout kas-rpi/kas-image-rpi02w.yml
```

This command:

- Clones required layers
- Generates `local.conf`
- Generates `bblayers.conf`

2.4 Start Build Container

```
kas/kas-container shell kas-rpi/kas-image-rpi02w.yml
```

2.5 Build Image

Inside the build environment:

```
bitbake custom-image
```

2.6 Output Image

```
build/tmp/deploy/images/rpi02w64/custom-image-rpi02w64.rootfs.wic
```

This image is flashed to the SD card.

2.7 Flash SD Card

```
sudo dd if=<IMAGE>.wic of=/dev/<your_device> bs=1MB conv=fsync
```

Install picocom:

```
sudo apt-get install picocom
```

Serial login output:

```
Vlab reference distribution 2026.06 rpi02w64 ttyS0

rpi02w64 login: root
```

Default credentials:

- Username: root
- Password: none

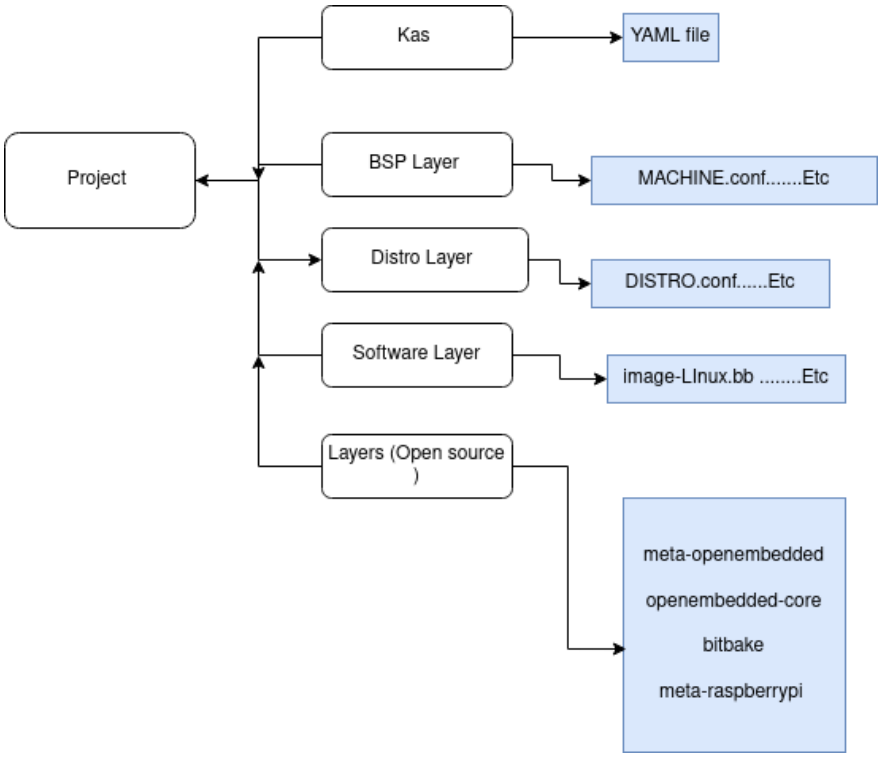
Chapter 3

Yocto Layer Architecture

3.1 Layer Overview

The project is structured into three main layers:

- **meta-rpi-bsp**: Board Support Package layer
- **meta-rpi-distro**: Distribution layer
- **meta-rpi-software**: Application/software layer



Chapter 4

BSP Layer

BSP(Board support Package)
Key components of a BSP Layer : A well -structured BSP Layer typically consists of the following components organized into a dedicated directory;

4.1 Machine Configuration

The machine configuration defines hardware parameters and boot settings:

```
require conf/machine/raspberrypi0-2w-64.conf

MACHINE_FEATURES = "apm usbhost vfat ext2 bluetooth wifi sdio"

PREFERRED_PROVIDER_virtual/kernel ?= "linux-raspberrypi"

ENABLE_UART = "1"

KERNEL_DEVICETREE = "\
broadcom/bcm2710-rpi-zero-2.dtb \
overlays/overlay_map.dtb \
"

IMAGE_FSTYPES = "wic ext4"
```

- To enable U-Boot as the bootloader for Raspberry Pi, you need to set the following variable:

```
RPI_USE_U_BOOT = "1"
```

4.2 Kernel Customization

Kernel configuration includes device tree selection and hardware-specific patches.

4.3 Bootloader

Bootloader configuration ensures correct system startup using U-Boot.

4.4 Drivers and Firmware

Includes hardware drivers and proprietary firmware required for device operation.

Chapter 5

Distro Layer

The distro layer defines global policies for the operating system.
The distribution configuration file needs to be created in the conf/distro directory of your layer. You need to name it using your distribution name (e.g. mydistro.conf).

5.1 Key Settings

Our configuration file needs to set the following required variables:

- DISTRO_NAME
- DISTRO_VERSION
- Init system: systemd
- Package format: ipk
- C library: glibc

  To prevent the installation of recommended Packages:
- Optimization: NO_RECOMMENDATIONS = "1"

Note: The DISTRO variable in your local.conf file determines the name of your distribution.

Chapter 6

Software Layer

This layer contains custom application recipes such as user applications and services.
For this project, we have developed a basic image that contains only essential components, such as the shell, basic commands, and BusyBox.

```
inherit core-image
#core-boot contains busybox ,and simple things..etc
IMAGE_INSTALL = " packagegroup-core-boot"
```

- Allow root login without a password (development only)

```
IMAGE_FEATURES = " allow-empty-password allow-root-login"
```

Chapter 7

Advanced Features

7.1 Partitioning A/B

Enables safe system updates by maintaining two system partitions.

7.2 OTA Updates

Implemented using SWUpdate for secure over-the-air updates.

7.3 Inter-Process Communication

D-Bus is added for service communication.

7.4 Container Support

LXC is integrated to support lightweight containerized environments.

Chapter 8

Conclusion

This project delivers a scalable Yocto-based embedded Linux system for Raspberry Pi Zero 2 W with OTA updates, container support, and industrial-grade architecture.